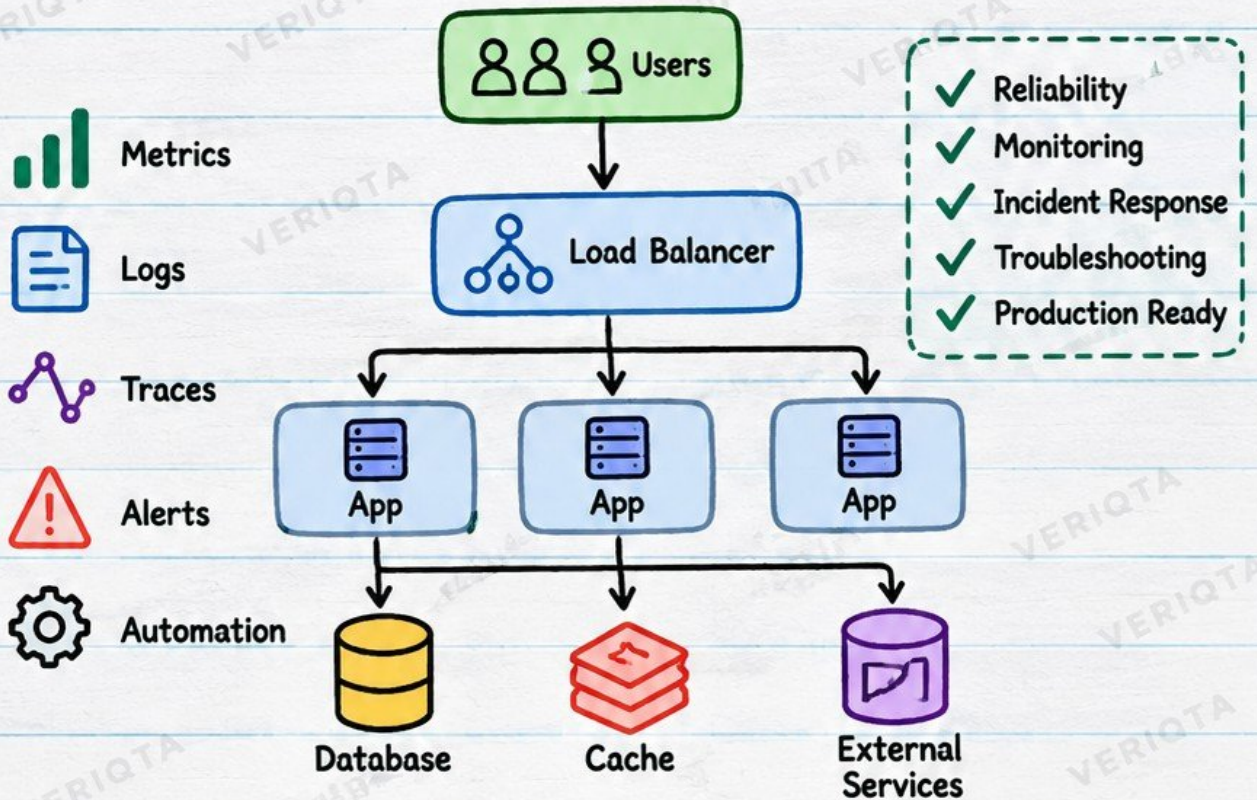


# SRE

# BIBLE

A PRACTICAL GUIDE FOR JUNIOR ENGINEERS



**BUILD | OPERATE | OBSERVE | TROUBLESHOOT**  
**IMPROVE | SCALE | KEEP SYSTEMS RELIABLE**

# What Is Site Reliability Engineering?

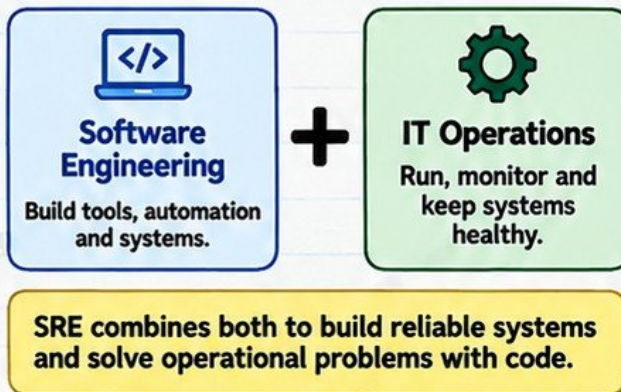
## 1. What Is SRE?

**Site Reliability Engineering (SRE)** is a discipline that uses software engineering to build and operate reliable, scalable and efficient systems.

## 2. Why Does SRE Exist?

- ✓ Modern applications are complex and always required to be available.
- ✓ Manual operations do not scale.
- ✓ SRE brings engineering practices to operations.
- ✓ It helps teams deliver new features without breaking reliability.

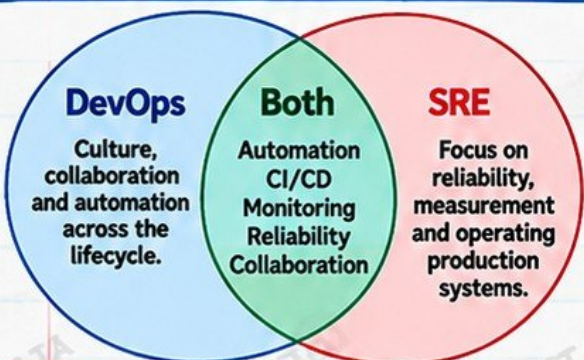
## 3. The Core Idea



## 4. Key Responsibilities

- Design reliable systems
- Monitor and detect issues
- Respond to and resolve incidents
- Automate repetitive operational tasks
- Improve performance and capacity
- Work with development teams
- Continuously improve reliability

## 5. Where SRE Fits



## 6. SRE vs DevOps vs Traditional Ops

| Aspect     | SRE                        | DevOps                       | Traditional Ops                  |
|------------|----------------------------|------------------------------|----------------------------------|
| Main Goal  | Reliability                | Faster delivery              | Keep systems running             |
| Approach   | Engineering and automation | Collaboration and automation | Manual processes                 |
| Focus Area | Production reliability     | Build, test and deploy       | Infrastructure management        |
| Mindset    | Measure and improve        | Break silos                  | Stability through manual control |

## 7. Real World Example

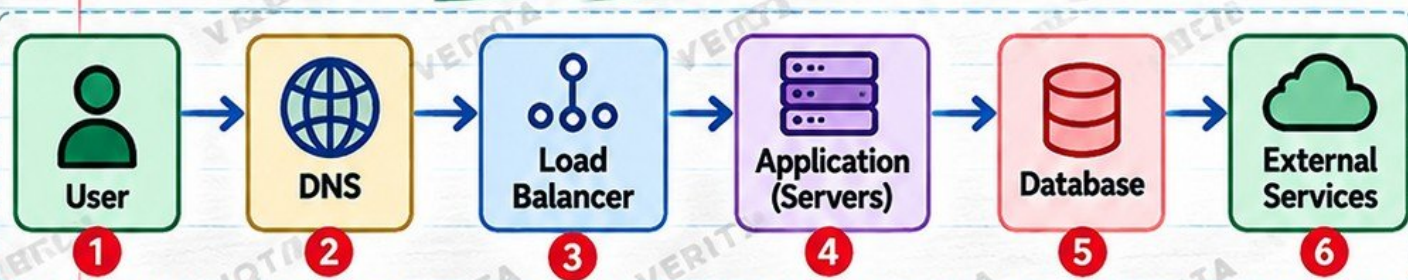


An SRE ensures this entire system is **reliable**, **observable** and can **recover quickly** when something fails.

## 8. Junior Engineer Takeaway

- SRE is not just fixing servers.
- It is a mindset of reliability, automation and continuous improvement.
- You use software engineering skills to solve real operational problems.
- SRE helps users have a better experience.

# How Production Systems Actually Work



## 1 User Sends a Request



- You type a URL in your browser.
- Your device sends a request to a domain name (for example, app.com).
- The request needs an IP address to find the server.

## 2 DNS Resolves the Domain



- DNS (Domain Name System) converts app.com to an IP address.
- Your device now knows which server to contact.
- If DNS fails, the application cannot be reached.

## 3 Load Balancer Receives the Request



- The request goes to a load balancer (for example, AWS ALB, NGINX).
- It distributes traffic across healthy application servers.
- It can also perform health checks, SSL termination and routing.

## 4 Application Handles the Request



- The load balancer forwards the request to one of the application servers.
- The application processes the request.
- It may read or write data from a database or call external services.

## 5 Database Stores or Retrieves Data



- The application queries a database (for example, PostgreSQL, MySQL).
- The database returns the requested data.
- If the database is slow or unavailable, the application will fail or be slow.

## 6 External Services and Dependencies



- The application may call other services such as payment, email, authentication, or third party APIs.
- If these dependencies are slow or down, the entire request can fail or be delayed.

## Where Failures Can Happen

|   |                   |                                             |
|---|-------------------|---------------------------------------------|
| 1 | User Device       | No internet, local network issue.           |
| 2 | DNS               | DNS outage, slow response, wrong record.    |
| 3 | Load Balancer     | Misconfiguration, unhealthy targets.        |
| 4 | Application       | High CPU, memory issue, crash, bug.         |
| 5 | Database          | Slow queries, high load, connection errors. |
| 6 | External Services | API downtime, rate limiting, network issue. |



## Key Takeaways

- ✓ A single user request travels through multiple components.
- ✓ Each component can fail and affect the entire system.
- ✓ SRE engineers understand the full flow to detect, troubleshoot and fix issues.
- ✓ We monitor each layer to keep the system reliable and available.



@veriqta



veriqta



veriqta



veriqta



veriqta



@veriqta\_academy

# Reliability, Availability & Durability

## 1. Reliability



Reliability means the system works correctly and consistently over time, doing what users expect.

### Example



An e-commerce site processes orders correctly, returns the right results and has minimal errors.

## 2. Availability



Availability means the system is accessible and able to respond when users need it.

### Example



A website is up and reachable 99.9% of the time, so users can access it.

## 3. Durability



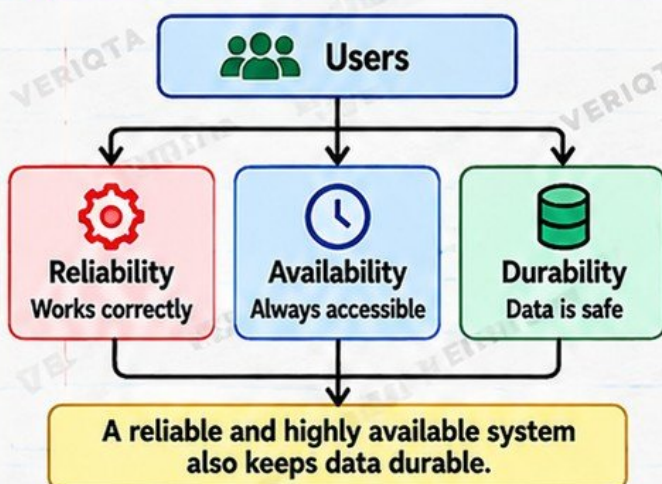
Durability means data is safely stored and can be recovered, even during failures such as server crashes.

### Example



User data, orders and transactions remain stored even if a server or disk fails.

## 4. How They Work Together



## 5. A System Can Be "Up" But Not Healthy



A system being up only means it is reachable. It does not automatically mean it is reliable or healthy.

### Example: Website is up but not Healthy



- ✗ Page loads but returns slow responses
- ✗ Some features return errors
- ✗ Database is overloaded
- ✗ Users experience timeouts
- ✗ Technically up, but not reliable

## 6. Real World Examples

| Scenario                         | Availability | Reliability | Durability | Explanation                                          |
|----------------------------------|--------------|-------------|------------|------------------------------------------------------|
| Website is down                  | ✗            | ✗           | ✓          | Not accessible to users.                             |
| Website is up but returns errors | ✓            | ✗           | ✓          | Users can access it, but it does not work correctly. |
| Website is slow                  | ✓            | ✗           | ✓          | Accessible, but poor user experience.                |
| Database lost data after crash   | ✗            | ✗           | ✗          | Not available, not reliable and data is lost.        |
| Website is up and working well   | ✓            | ✓           | ✓          | Accessible, correct results and data is safe.        |



## 7. Key Takeaways

- 1 Reliability means the system works correctly over time.
- 2 Availability means the system is accessible when users need it.
- 3 Durability means data is safely stored even during failures.
- 4 A system can be up but still unhealthy.
- 5 SRE engineers care about all three to provide a good user experience.
- 6 You need all three for a truly reliable production system.



@veriqta



veriqta



veriqta



veriqta



veriqta



@veriqta\_academy

# SLIs: What Should We Measure?

A Service Level Indicator (SLI) is a measurable signal that shows how well a system is meeting user needs.  
Good reliability starts with **measuring what users actually experience**.

## 1. Availability



- Percentage of time the service is accessible and working.
- Shows if users can reach the service.

### Example

99.9% of API requests are successful.

## 2. Latency



- Time taken to respond to a request.
- Shows how fast users get a response.

### Example

95% of requests respond in less than 300 ms.

## 3. Error Rate



- Percentage of requests that fail.
- Shows if users are experiencing errors.

### Example

Less than 0.1% of requests return errors (4xx or 5xx).

## 4. Throughput



- Number of requests or transactions handled in a period of time.
- Shows how much traffic the service can handle.

### Example

Handle 1,000 requests per second.

## 5. Durability



- Ability to safely store data and prevent data loss, even during failures.
- Shows if user data is protected.

### Example

No data loss during server crashes or restarts.

## 6. Freshness



- How up to date the data is.
- Shows if users are seeing recent and correct information.

### Example

New data is visible to users within 5 minutes.

## 7. Why These SLIs Matter

- ✓ They measure what users actually experience.
- ✓ They help you detect problems early.
- ✓ They provide objective data for decision making.
- ✓ They are used to set Service Level Objectives (SLOs).
- ✓ They help improve reliability over time.

## 8. Real World Example (E-commerce)

| SLI          | What We Measure      | Example Target |
|--------------|----------------------|----------------|
| Availability | Successful requests  | 99.9%          |
| Latency      | Response time (95th) | < 300 ms       |
| Error Rate   | Failed requests      | < 0.1%         |
| Throughput   | Requests per second  | 1,000 rps      |
| Durability   | No data loss         | 0 data loss    |
| Freshness    | Data update delay    | < 5 minutes    |



@veriqta



veriqta



veriqta



veriqta



veriqta



@veriqta\_academy

# SLOs: How Reliable Should We Be?

A Service Level Objective (SLO) is a target level of reliability for a specific user facing metric (SLI) over a time period.

**It defines how reliable we want to be and is based on what users actually experience.**

## 1. SLO Structure



**SLI**

What we measure  
(for example successful requests).



**Target**

How reliable we want to be  
(for example 99.9%).



**Measurement Window**

The time period to evaluate  
(for example 30 days).

## 2. Example SLO



**99.9% successful requests over 30 days**

- This means at least 99.9% of user requests must be successful.
- No more than 0.1% of requests can fail.
- Measured over a full 30 day period.
- This is based on what users experience, not just whether servers are running.

## 3. What Does 99.9% Actually Mean?

| Time Period | Allowed Downtime (0.1%) | Equivalent    |
|-------------|-------------------------|---------------|
| 1 day       | 1.44 minutes            | 86 seconds    |
| 7 days      | 10.1 minutes            | 607 seconds   |
| 30 days     | 43.2 minutes            | 2,592 seconds |
| 90 days     | 2.16 hours              | 7,776 seconds |

## 4. Visual Example (100,000 Requests)

100,000 total requests



## 5. Real World Example



**E-commerce Website**

SLO: 99.9% successful requests over 30 days

- If the site receives 100,000 requests in 30 days, no more than 100 can fail.
- Users will see a fast and reliable experience even if a small number of requests fail.
- This accounts for real world issues such as network problems, dependency failures and maintenance events.

## 6. Common Mistakes

- ✗ Measuring server uptime instead of user experience.
- ✗ Using an SLO that is too aggressive.
- ✗ Not defining a clear measurement window.
- ✗ Changing the SLO frequently.
- ✗ Ignoring important user journeys (for example checkout).
- ✗ Setting the same SLO for non critical and critical services.

## 7. Key Takeaways



- ✓ SLOs define the reliability target for user experience.
- ✓ They are based on SLIs, not server uptime.
- ✓ They include a clear target and measurement window.
- ✓ They help teams make better reliability and product decisions.
- ✓ A small amount of failure is expected and normal.



@veriqta



veriqta



veriqta



veriqta



veriqta



@veriqta\_academy

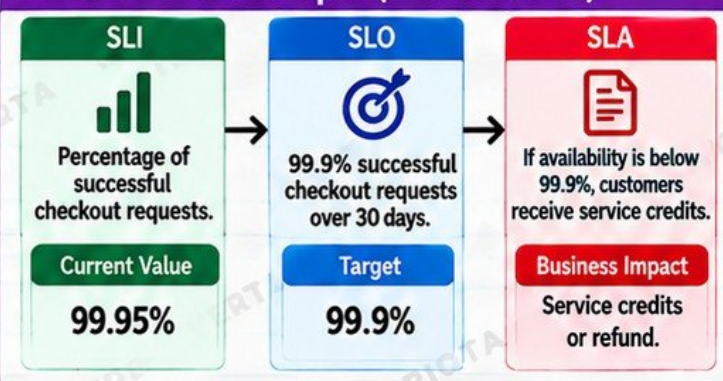
# SLAs vs SLOs vs SLIs

**SLIs, SLOs and SLAs are connected but serve different purposes.**  
SLIs measure, SLOs set targets, and SLAs turn those targets into business commitments.



| Aspect                 | SLI<br>(Service Level Indicator)     | SLO<br>(Service Level Objective)                      | SLA<br>(Service Level Agreement)                                          |
|------------------------|--------------------------------------|-------------------------------------------------------|---------------------------------------------------------------------------|
| What it is             | A measurement of system performance. | A target for the SLI (for example 99.9%).             | A formal commitment to customers based on SLOs.                           |
| Main purpose           | Measure what is happening.           | Define how reliable we want to be.                    | Set expectations with customers.                                          |
| Audience               | Engineering teams.                   | Engineering and product teams.                        | Business, legal and customers.                                            |
| Example                | Percentage of successful requests.   | 99.9% successful requests over 30 days.               | If availability is below 99.9%, service credits may apply.                |
| Focus                  | Current performance.                 | Target reliability level.                             | Contract and consequences.                                                |
| Time period            | Measured continuously.               | Evaluated over a period (for example 30 days).        | Usually longer term (for example monthly or yearly).                      |
| Consequence if not met | None directly (it is just a metric). | Internal action (improve reliability, reduce errors). | Business consequences (for example service credits, financial penalties). |

## Real World Example (E-commerce)



## Why Engineers Use SLOs Internally

- ✓ Helps focus on user experience.
- ✓ Provides a clear reliability target.
- ✓ Guides engineering and operational decisions.
- ✓ Allows healthy trade-offs between reliability and speed.

## Why SLAs Have Business Consequences

- ✓ They are formal commitments to customers.
- ✓ They set expectations for service level and support.
- ✓ If the SLA is not met, the business may owe service credits or face penalties.
- ✓ They are used in contracts and customer agreements.



## Key Takeaways

- 1 SLIs measure real performance.
- 2 SLOs set the target for reliability.
- 3 SLAs turn reliability into a business commitment.
- 4 Engineers focus on SLIs and SLOs to improve systems.
- 5 SLAs protect customers and create business accountability.
- 6 All three work together to deliver reliable services and user trust.



@veriqta



veriqta



veriqta



veriqta



veriqta



@veriqta\_academy

# Error Budgets

An error budget is the amount of unreliability a service is allowed to have, based on its SLO. It helps teams **balance reliability** with **shipping new features**.

## 1. Formula

$$\text{Error Budget} = 100\% - \text{SLO}$$

If your SLO is 99.9%, your error budget is 0.1%.

## 2. What It Means

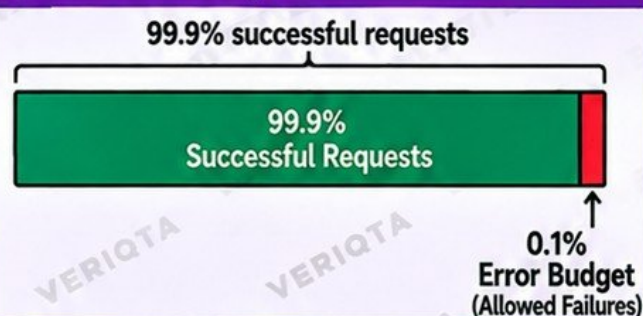


- The error budget is the percentage of user requests that can fail while still meeting the SLO.
- It gives teams a clear limit for how much risk they can take.

## 3. Example Calculation (99.9% SLO)

| Time Period | Total Requests | Allowed Failures (0.1%) |
|-------------|----------------|-------------------------|
| 1 day       | 100,000        | 100                     |
| 7 days      | 1,000,000      | 1,000                   |
| 30 days     | 10,000,000     | 10,000                  |

## 4. Visual Example



## 5. How Teams Use Error Budgets



### If we are within the error budget

We can take more risk and ship new features faster.



### If we are close to exhausting it

We reduce risk, focus on stability and fix issues before shipping new features.



### If we exceed the error budget

We must stop risky changes and focus on improving reliability.



### It helps with data driven decisions

It removes opinion and gives a clear, measurable limit.

## 6. Real World Scenario

| Scenario                             | Error Budget Status | What the Team Does                    |
|--------------------------------------|---------------------|---------------------------------------|
| Service is stable and within budget  | ✓                   | Continue shipping new features.       |
| Using most of the budget             | ⚠                   | Be more careful, reduce risk.         |
| Budget exceeded due to many failures | ✗                   | Stop new features and focus on fixes. |

## 7. Key Takeaways



- ✓ Error budget = 100% - SLO.
- ✓ It is the allowed amount of unreliability.
- ✓ It helps balance reliability with feature delivery.
- ✓ It guides when to take risk and when to focus on stability.
- ✓ It leads to better decisions and more reliable systems.



@veriqta



veriqta



veriqta



veriqta



veriqta



@veriqta\_academy

# The Four Golden Signals

The four golden signals give a quick and reliable view of a system's health. They help SREs understand what is happening, **find the real problem faster** and make better decisions.

## 1. Latency



- Time taken to respond to a request.
- Shows how fast users get a response.
- High latency means requests are taking too long.

### Example

API response time in milliseconds



## 2. Traffic



- Number of requests over time.
- Shows how much load the system is handling.
- A sudden increase or decrease can indicate a problem or an event.

### Example

Requests per second



## 3. Errors



- Percentage of requests that fail.
- Shows if users are experiencing errors.
- High error rate means something is not working correctly.

### Example

Error rate (percentage)



## 4. Saturation



- How much of your resources are being used (CPU, memory, disk, network, etc).
- Shows if the system is under heavy load.
- High saturation means the system is running out of resources.

### Example

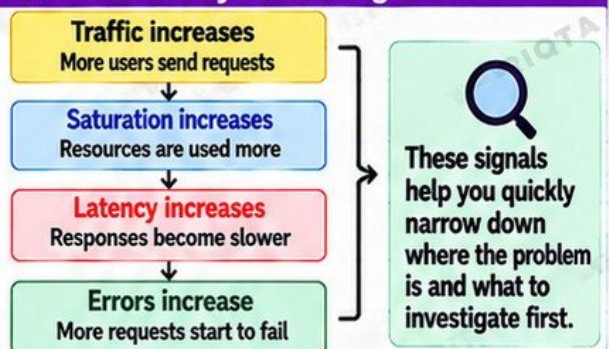
CPU usage (percentage)



## 5. What Each Signal Tells You

| Signal     | Tells You                         | Common Questions                                    |
|------------|-----------------------------------|-----------------------------------------------------|
| Latency    | How fast users get a response.    | Are requests slow? Where is the delay?              |
| Traffic    | How much load the system handles. | Is traffic higher or lower than usual?              |
| Errors     | If requests are failing.          | Are users seeing errors? What type of errors?       |
| Saturation | If the system is under pressure.  | Are we running out of CPU, memory, disk or network? |

## 6. How They Work Together



## 7. Real World Example

A website suddenly becomes slow.



- Latency is high
- Traffic is normal
- Errors are low
- Saturation on the database is high



This suggests the database is a bottleneck.

## 8. Key Takeaways



- ✓ Latency, traffic, errors and saturation give a clear view of system health.
- ✓ They help you find problems faster.
- ✓ Always look at all four signals together.
- ✓ They show symptoms, then you investigate the cause.
- ✓ They are the starting point for most production issues.



@veriqta



veriqta



veriqta



veriqta



veriqta



@veriqta\_academy

# Metrics, Logs & Traces

Metrics, logs and traces give different views of your system.  
Engineers use **all three together** to understand what is happening in production.

## 1. Metrics



- Numerical data over time.
- Shows system health and trends.
- Answers higher level questions like how much, how many and how fast.

### Example

Request count (per minute)



## 2. Logs



- Text records of events.
- Shows detailed information about what happened.
- Answers questions like what exactly happened and why did it happen.

### Example

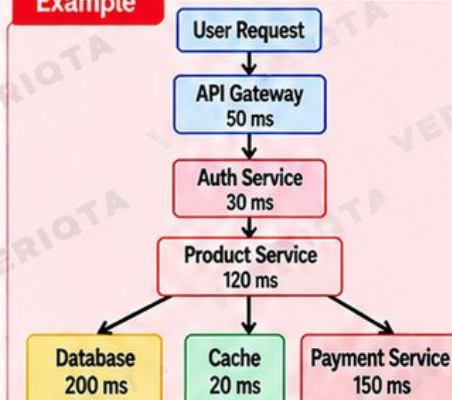
```
2026-10-07 10:15:23 INFO User login successful
2026-10-07 10:15:24 WARN Slow database query
2026-10-07 10:15:25 ERROR Payment failed
2026-10-07 10:15:26 INFO Retrying request
```

## 3. Traces



- Shows the journey of a single request across services.
- Visualizes how long each service takes.
- Answers questions like where is the delay and which service is failing.

### Example



## 4. When to Use Each One

| Use Case                            | Metrics | Logs | Traces |
|-------------------------------------|---------|------|--------|
| Check overall system health         | ✓       |      |        |
| Find exact error details            |         | ✓    |        |
| Understand a slow request           |         |      | ✓      |
| See trends over time                | ✓       |      |        |
| Investigate a specific user issue   |         | ✓    | ✓      |
| Find which service is causing delay |         |      | ✓      |

## 5. How They Work Together



1. Start with Metrics  
Find the problem (something looks unusual)



2. Use Traces  
Find where the problem is (which service is slow or failing)



3. Check Logs  
Find the exact reason (error message or event)

Using all three together gives a complete picture and helps you solve production problems faster and with more confidence.

## 6. Real World Example

Users report the website is slow

| Metrics                                                                                                                                                                                                                                                                      | Traces                                                                                                                                                                                                                                                                                             | Logs                                                                                                                                                                                    |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <ul style="list-style-type: none"> <li>• Latency is high.</li> <li>• Error rate is normal.</li> <li>• Traffic is normal.</li> <li>• This shows there is a performance issue.</li> </ul> | <ul style="list-style-type: none"> <li>API Gateway 50 ms</li> <li>Auth Service 70 ms</li> <li>Product Service 600 ms</li> <li>Database 50 ms</li> </ul> <ul style="list-style-type: none"> <li>• Traces show the Product Service is slow.</li> <li>• This tells us where to focus next.</li> </ul> | <div>ERROR</div> <p>Database connection timeout</p> <ul style="list-style-type: none"> <li>• Logs show database connection timeouts.</li> <li>• This gives the exact reason.</li> </ul> |

## 7. Key Takeaways



- ✓ Metrics show the big picture.
- ✓ Logs show the details.
- ✓ Traces show the full request journey.
- ✓ They answer different questions.
- ✓ Using all three together helps you find and fix problems faster.



@veriqta



veriqta



veriqta



veriqta



veriqta



@veriqta\_academy

# Monitoring vs Alerting

Monitoring observes your systems. Alerts demand attention.  
Good alerting helps you respond to **real problems quickly without creating noise.**

## 1. What Is Monitoring?



- Continuously collects data from your systems.
- Helps you understand how your systems are behaving.
- Used to visualize trends, capacity, performance and health.

**Examples:** Metrics, logs, traces, dashboards

## 2. What Is Alerting?



- Notifies you when something needs attention.
- Focused on important issues that can affect users or the business.
- Should be actionable so the team knows what to do next.

**Examples:** Critical errors, service down, high latency, high failure rate

## 3. Key Differences

| Aspect    | Monitoring                | Alerting                                            |
|-----------|---------------------------|-----------------------------------------------------|
| Purpose   | Observe and understand    | Demand attention                                    |
| Data      | Metrics, logs, traces     | Uses thresholds or conditions from data             |
| Output    | Dashboards and graphs     | Notifications (for example Slack, email, PagerDuty) |
| Focus     | Trends and overall health | Current and urgent issues                           |
| Audience  | Engineers, stakeholders   | On call engineers                                   |
| Frequency | Always on                 | Only when needed                                    |

## 4. Symptoms vs Causes

### Symptom



- High error rate
- High latency
- Service is down
- High CPU usage
- Database connection errors

### Cause



- Bug in the application
- Database overload
- Network issue
- Dependency failure
- Misconfiguration
- Resource exhaustion

Alerts usually tell you about **symptoms**.  
You still need to investigate to find the **root cause**.

## 5. Why Alerting on Everything Creates Noise



- ✗ Too many alerts lead to alert fatigue.
- ✗ Important alerts get ignored.
- ✗ Teams lose trust in the alerting system.
- ✗ It makes it harder to find real issues.
- ✗ It increases stress for on call engineers.

## 6. Characteristics of a Good Alert

- ✓ Actionable and clear.
- ✓ Focused on user impact.
- ✓ Low false positives.
- ✓ Includes useful context (for example affected service).
- ✓ Goes to the right people.
- ✓ Does not alert on routine or non critical issues.

## 7. Examples

| Good Alert                      | Bad Alert                     |
|---------------------------------|-------------------------------|
| ✓ High error rate for 5 minutes | ✗ CPU above 70 percent        |
| ✓ API latency above 2 seconds   | ✗ Memory above 60 percent     |
| ✓ Service is down               | ✗ Single error in logs        |
| ✓ Database connection failures  | ✗ Minor latency increase      |
| ✓ Disk usage above 90 percent   | ✗ Alerts for every deployment |

## 8. Key Takeaways



- ✓ Monitoring gives you visibility.
- ✓ Alerting tells you when to act.
- ✓ Alert on important symptoms, not everything.
- ✓ Good alerts are actionable and reduce noise.
- ✓ Always investigate to find the root cause.
- ✓ The goal is a reliable system and a calm on call team.



@veriqta



veriqta



veriqta



veriqta



veriqta

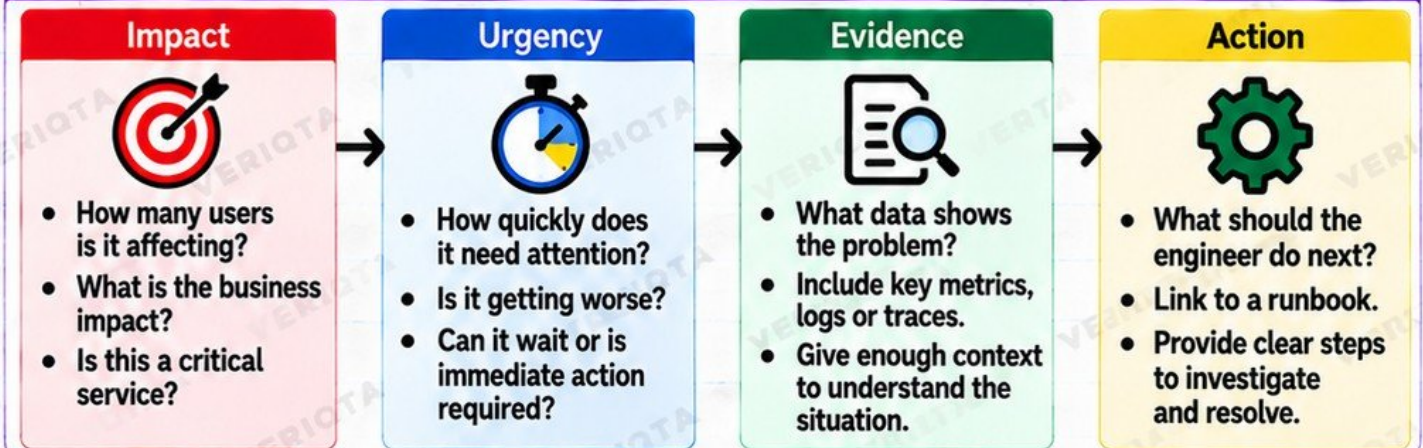


@veriqta\_academy

# What Makes a Good Alert?

A good alert is clear, actionable and focused on real impact. It helps the right people respond quickly and **reduces noise**.

## 1. The Four Key Elements



## 2. Good vs Bad Alert Example

| ✓ Good Alert                                                            | ✗ Bad Alert                       |
|-------------------------------------------------------------------------|-----------------------------------|
| High error rate on payment service                                      | Server CPU high                   |
| Impact: Affects ~30% of user checkouts                                  | Impact: Not clear                 |
| Urgency: High (increasing)                                              | Urgency: Not clear                |
| Evidence: Error rate 12% (threshold 5%) for 10 minutes<br>Graph: <link> | Evidence: CPU at 75% (no context) |
| Action: Check runbook<br><runbook link>                                 | Action: Not clear (no next steps) |

## 3. Alert Fatigue and Noisy Alerts

|                                                                                                                                                                                                                                                                                                                                                                 |                                                                                                                                                                                                                                                                                                                             |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <b>Alert Fatigue</b> <ul style="list-style-type: none"> <li>Too many alerts make engineers ignore important ones.</li> <li>It leads to slower response times and real issues being missed.</li> <li>It creates stress and reduces trust in the alerting system.</li> </ul> |  <b>Noisy Alerts</b> <ul style="list-style-type: none"> <li>Alerts that fire too often or on low impact issues.</li> <li>Usually caused by poor thresholds, lack of context or alerting on symptoms that are not actionable.</li> </ul> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 4. Actionable Thresholds

|                                                                                                                                                                                                                                                                                                                                                                                                                      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <ul style="list-style-type: none"> <li>Use thresholds based on real user impact.</li> <li>Avoid alerting on normal or expected behaviour.</li> <li>Use time windows (for example 5 minutes) to avoid short spikes.</li> <li>Tune thresholds based on historical data.</li> <li>Alert on symptoms that require action.</li> </ul> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 5. Include a Runbook

|                                                                                                                                                                                                                                                                                                                                             |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <ul style="list-style-type: none"> <li>Link to a runbook in every alert.</li> <li>Explain what the alert means.</li> <li>List immediate checks to perform.</li> <li>Include escalation steps.</li> <li>Keep it short, clear and up to date.</li> </ul> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 6. Who Should Be Alerted?

|                                                                                                                                                                                                                                                                                                                            |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <ul style="list-style-type: none"> <li>Only alert people who can take action.</li> <li>Route to the right team based on the service.</li> <li>Use escalation for ongoing critical issues.</li> <li>Avoid alerting everyone.</li> </ul> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 7. Key Takeaways

|                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <ul style="list-style-type: none"> <li>✓ A good alert has clear impact, urgency, evidence and action.</li> <li>✓ Reduce noise to avoid alert fatigue.</li> <li>✓ Use actionable thresholds based on real user impact.</li> <li>✓ Always include a runbook.</li> <li>✓ Good alerts help you respond faster and keep systems reliable.</li> </ul> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

# Health Checks & Probes

Health checks help determine if a service is healthy and ready to serve traffic.  
A process running **does not always mean** the application can actually handle requests.

## 1. Liveness Check



- Checks if the application is still running.
- If it fails, the container is restarted.
- Answers the question: Is the process alive?

### Example

- /health/live endpoint returns 200 OK
- If it fails, restart the pod or container.

## 2. Readiness Check



- Checks if the application is ready to serve traffic.
- If it fails, the pod is removed from the load balancer.
- Answers the question: Can this instance handle requests?

### Example

- /health/ready endpoint returns 200 OK
- If it fails, stop sending traffic to this instance.

## 3. Startup Check



- Gives the application time to start up.
- Prevents restarts during initialization.
- Used for applications that take time to load (dependencies, cache, large data, etc).

### Example

- /health/startup endpoint returns 200 OK
- Only after it passes, liveness and readiness checks start.

## 4. Dependency Health



- Checks the health of external dependencies such as databases, queues, APIs and caches.
- The application may be running, but it cannot serve traffic if a critical dependency is down.

### Example

- Check database connection
- Check external API health
- Check message queue health
- Return failure if a critical dependency is unavailable.

## 5. Process Running ≠ Application Healthy

### Process Running



- The application process is alive.
- It is using CPU and memory.
- It has not crashed.

≠

### Application Can Be Unhealthy



- Database connection failed.
- External API is down.
- Cache is unavailable.
- The application is returning errors.
- It cannot handle user requests even though the process is running.

## 6. Real World Example



## 7. Common Health Check Types

| Type       | Purpose                        | Failure Action                           |
|------------|--------------------------------|------------------------------------------|
| Liveness   | Is the process alive?          | Restart the container                    |
| Readiness  | Can it serve traffic?          | Remove from load balancer                |
| Startup    | Has it finished starting?      | Wait before enabling other checks        |
| Dependency | Are external services healthy? | Mark as unhealthy or remove from traffic |

## 8. Good Practices

- Check real application functionality.
- Include critical dependencies.
- Use separate endpoints for liveness, readiness and startup.
- Set appropriate timeouts and thresholds.
- Avoid heavy or expensive checks.
- Keep health check logic simple and fast.
- Monitor health check results.

## 9. Key Takeaways

- Health checks help keep unhealthy instances out of traffic.
- Liveness, readiness and startup checks solve different problems.
- A running process does not mean the application can serve requests.
- Check critical dependencies.
- Good health checks improve reliability and user experience.

# Timeouts, Retries & Backoff

Distributed requests can and will fail. Timeouts, retries and backoff help your systems handle failures, but they **must be used correctly** to avoid making an outage worse.

## 1. Why Distributed Requests Fail



### Requests can fail because of:

- Network issues and packet loss
- High latency
- Service overload
- Dependencies being down
- Resource exhaustion (CPU, memory, DB)
- Bugs and unexpected errors
- Slow responses that exceed time limits

## 2. Timeouts



- A timeout sets a maximum amount of time to wait for a response.
- Prevents requests from hanging forever.
- Helps free up resources and keeps your system responsive.
- Should be set based on realistic latency (for example p95 or p99).

### Example

Timeout: 2 seconds  
If no response in 2 seconds, return an error and move on.

## 3. Retries



- A retry tries the request again when it fails.
- Helps with temporary failures such as network blips or brief overload.
- Should only be used for safe operations (for example GET or idempotent requests).
- Must have a limit on the number of retries.

### Example

Retry up to 3 times  
Only retry on specific errors (for example 5xx or timeout).

## 4. Exponential Backoff

- Wait for a longer time between retries instead of retrying immediately.
- Reduces load on a struggling service.
- Helps the service recover.
- Often combined with jitter to avoid many clients retrying at the same time.

### Example Backoff Sequence (base 500 ms)



## 5. Backoff with Jitter

- Jitter adds a small random delay to each retry.
- Prevents a thundering herd where many clients retry at the same time.
- Results in more stable recovery during an outage.

### Example (with jitter)

Instead of always waiting 2s, wait a random time between 1.5s and 2.5s.



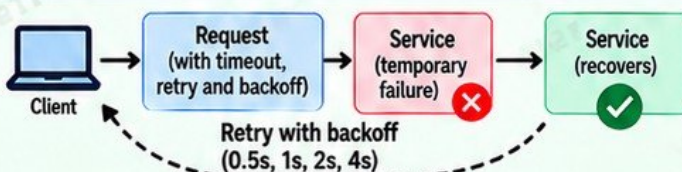
## 6. When Retries Can Make an Outage Worse

- Too many retries increase load on an already failing service.
- Can exhaust resources and make recovery slower.
- Can cause cascading failures across dependent services.
- Can hide real problems and make debugging harder.

### Avoid these problems

- ✗ Do not use unlimited retries.
- ✗ Use exponential backoff with jitter.
- ✗ Retry only on specific, transient errors.
- ✗ Set reasonable timeouts.
- ✗ Monitor retry rates and failures.

## 7. Real World Example



## 8. Key Takeaways



- ✓ Set timeouts to avoid hanging requests.
- ✓ Use retries only for safe and idempotent operations.
- ✓ Use exponential backoff with jitter.
- ✓ Do not use unlimited retries.
- ✓ Be careful with retries during an outage.
- ✓ Tune values based on real latency and service behaviour.
- ✓ Monitor timeouts, retries and failure rates.



@veriqta



veriqta



veriqta



veriqta



veriqta



@veriqta\_academy

# Redundancy & High Availability

Redundancy removes single points of failure and keeps your service running when something fails. High availability uses redundancy and failover to minimize downtime, but **redundancy alone does not guarantee reliability.**

## 1. Single Point of Failure (SPOF)

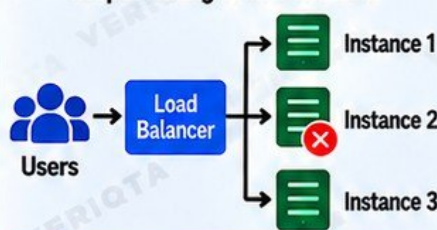
If one component fails, the entire service goes down.



- ✗ One server
- ✗ One database
- ✗ One network path
- ✗ One availability zone
- ✗ One third party dependency

## 2. Multiple Instances

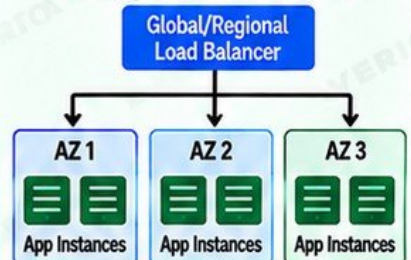
Run multiple instances so the service keeps running even if one fails.



- ✓ Distribute traffic across multiple instances
- ✓ If one instance fails, others keep serving
- ✓ Handle higher traffic
- ✓ Enable rolling updates without downtime

## 3. Availability Zones

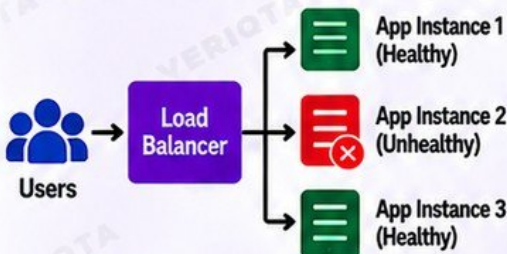
Spread resources across multiple availability zones or data centers.



- ✓ Survives a full zone failure
- ✓ Reduces impact of regional issues
- ✓ Keeps service available to users
- ✓ Often used with managed services

## 4. Load Balancing

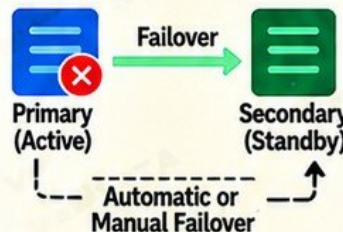
A load balancer distributes traffic across healthy instances and removes unhealthy ones.



- ✓ Distributes traffic evenly
- ✓ Performs health checks
- ✓ Stops sending traffic to unhealthy instances
- ✓ Helps with scaling and high availability

## 5. Failover

When the primary system fails, traffic is automatically or manually switched to a backup.



- ✓ Detects failure quickly
- ✓ Switches traffic to a healthy backup
- ✓ Can be automatic or manual
- ✓ Used for databases, entire regions or critical services

## 6. Redundancy ≠ Reliability

Redundancy reduces the chance of failure, but it does not guarantee reliability.

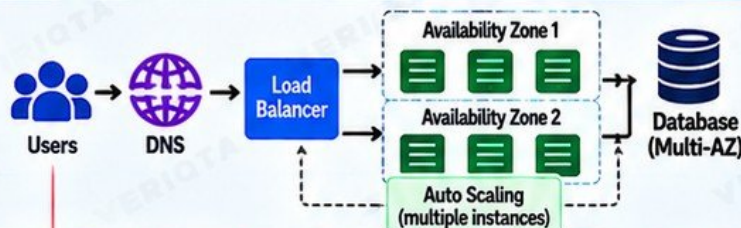
You can still have issues due to:

- ✗ Misconfigurations across instances
- ✗ Shared dependencies (for example a single database)
- ✗ Bugs in the application
- ✗ Cascading failures
- ✗ Resource exhaustion
- ✗ Network issues
- ✗ Bad deployments

Reliability also requires:

- ✓ Proper monitoring and alerting
- ✓ Testing failover regularly
- ✓ Handling errors and timeouts correctly
- ✓ Good operational practices

## 7. Real World Example (Web Application)



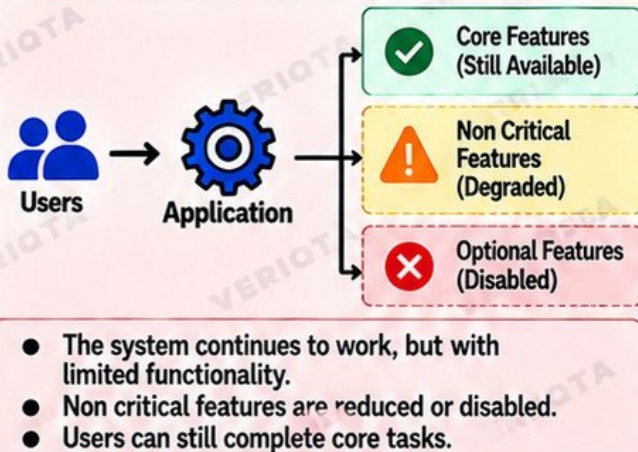
## 8. Key Takeaways

- ✓ Remove single points of failure.
- ✓ Use multiple instances and load balancing.
- ✓ Spread resources across availability zones.
- ✓ Implement failover for critical components.
- ✓ Test your failover and recovery regularly.
- ✓ Redundancy alone does not guarantee reliability.
- ✓ Combine redundancy with monitoring, good design and operational discipline.

# Graceful Degradation & Resilience

Graceful degradation allows a system to continue providing reduced functionality when a dependency fails, instead of completely going down. This improves user experience and makes the system **more resilient**.

## 1. What is Graceful Degradation?



## 2. Graceful Degradation vs Complete Failure

### Complete Failure



- Service is completely down
- All features unavailable
- Users cannot perform any actions
- Higher business impact
- Worse user experience

### Graceful Degradation



- Service continues to run
- Core features remain available
- Non critical features disabled or simplified
- Users can still complete important tasks
- Lower business impact
- Better user experience

## 3. Real World Example (E Commerce)



## 4. Techniques for Graceful Degradation

|                                                                                                                   |                                                                     |
|-------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
|  <b>Fallbacks</b>              | Use default values or simpler alternatives when a dependency fails. |
|  <b>Cached Data</b>            | Serve cached or stale data instead of failing.                      |
|  <b>Feature Flags</b>          | Disable non critical features during an incident.                   |
|  <b>Timeouts &amp; Retries</b> | Use timeouts and limited retries to avoid hanging.                  |
|  <b>Circuit Breakers</b>       | Stop calling a failing service to prevent cascading failures.       |
|  <b>Load Shedding</b>          | Reduce non essential load when the system is under stress.          |

## 5. Resilience Patterns



## 7. Benefits

- 
- Better user experience during incidents
  - Lower business impact
  - More resilient and stable systems
  - Prevents total service outages
  - Gives time to fix the real problem

## 6. What to Degradate First?

- ✓ Debug or admin features
- ✓ Recommendations and personalization
- ✓ Search auto complete
- ✓ Non critical third party integrations
- ✓ Real time features (for example live chat, live analytics)
- ✓ High resource features (for example large reports)
- ✗ Core features (should stay available if possible)

## 8. Key Takeaways

- 
- ✓ Graceful degradation keeps core functionality available.
  - ✓ Not all features have the same importance.
  - ✓ Use proven resilience patterns like caching, circuit breakers and feature flags.
  - ✓ Plan for dependency failures.
  - ✓ A resilient system degrades gracefully instead of completely failing.

# Incident Response

Incident response is a structured process to detect, assess, mitigate, communicate, recover and verify when something goes wrong in production. The goal is to **restore service quickly** and minimize impact on users and the business.

## 1. The Incident Response Process



## 2. Incident Severity Levels

| Severity | Impact                                                                                                                   | Examples                                                                                                          |
|----------|--------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| SEV 1    | <ul style="list-style-type: none"> <li>Service down or major outage.</li> <li>Large number of users affected.</li> </ul> | <ul style="list-style-type: none"> <li>Complete service outage</li> <li>Core functionality not working</li> </ul> |
| SEV 2    | <ul style="list-style-type: none"> <li>Major degradation.</li> <li>Important features impacted.</li> </ul>               | <ul style="list-style-type: none"> <li>High error rates</li> <li>Significant performance issues</li> </ul>        |
| SEV 3    | <ul style="list-style-type: none"> <li>Minor impact.</li> <li>Limited number of users affected.</li> </ul>               | <ul style="list-style-type: none"> <li>Non critical feature issues</li> <li>Workaround available</li> </ul>       |
| SEV 4    | <ul style="list-style-type: none"> <li>Minimal impact.</li> <li>No user impact or very low impact.</li> </ul>            | <ul style="list-style-type: none"> <li>Cosmetic issues</li> <li>Internal system issues</li> </ul>                 |

## 3. Role of the Incident Commander



- ✓ Owns the incident end to end.
- ✓ Coordinates the response.
- ✓ Makes decisions and keeps the team focused.
- ✓ Ensures clear communication.
- ✓ Focuses on restoring service quickly.
- ✓ Brings in the right people when needed.

## 4. Communication During an Incident



- ✓ Keep updates short, clear and frequent.
- ✓ Share what is known, what is being done, and what is next.
- ✓ Communicate with internal teams and external users if needed.
- ✓ Avoid speculation.
- ✓ Use a consistent channel and a single source of truth.

## 5. Restore Service Before Deep Investigation



During an incident, the priority is to restore service first. Deep investigation and root cause analysis can happen after the service is stable.

- ✓ Get users working again as quickly as possible.
- ✓ Use workarounds, failover or rollback if needed.
- ✓ Avoid spending too much time on perfect root cause during the incident.
- ✓ Collect information for later analysis.

## 6. After the Incident



- ✓ Conduct a blameless post incident review.
- ✓ Find the root cause.
- ✓ Identify improvements to prevent recurrence.
- ✓ Update runbooks and monitoring.
- ✓ Share learnings with the team.

## 7. Key Takeaways



- ✓ Have a clear incident response process.
- ✓ Detect and act quickly.
- ✓ Communicate early and often.
- ✓ Focus on restoring service first.
- ✓ Use defined severity levels.
- ✓ The incident commander keeps the response organized.
- ✓ Learn from every incident.
- ✓ A good incident response reduces downtime and builds more reliable systems.

## 8. Follow VERIQTA for More DevOps and SRE Content



@veriqta



veriqta



veriqta



veriqta



veriqta



@veriqta\_academy

# Troubleshooting a Production Incident

Use a clear and structured approach to troubleshoot production issues.  
Here is a real world example of an **API latency suddenly increasing**.

## 1. Symptom

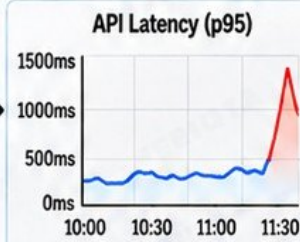
What is happening?



- API latency suddenly increased.
- Users are experiencing slow responses.
- Error rate may also start to increase.

## 2. Evidence

What do the signals show?



- p95 latency increased from 200ms to 1200ms.
- Traffic looks normal.
- Some timeouts and increased 5xx errors.

## 3. Hypothesis

What could be causing it?



- 1 Database is slow.
- 2 External dependency is slow.
- 3 High CPU or memory on the application.
- 4 Network or load balancer issue.
- 5 Recent deployment introduced a bug.

## 4. Diagnostic Check

What do we check?

-  Check application metrics (CPU, memory, threads).
-  Check database metrics (latency, connections).
-  Check external API latency and error rates.
-  Check load balancer and network metrics.
-  Check application logs and traces for slow queries or errors.

## 5. Cause

What is the root cause?



- Database query latency is very high.
- A recent query is scanning a large table without an index.
- This is causing the API to respond slowly.

## 6. Fix

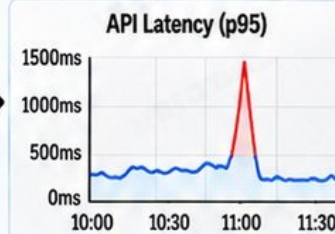
What do we do to resolve it?



- Add the missing index on the database table.
- Optimize the slow query.
- Restart affected application instances if needed.
- Monitor to confirm latency decreases.

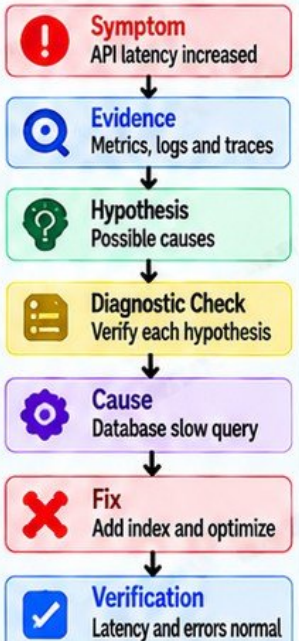
## 7. Verification

Is the issue resolved?



- Latency back to normal (around 200ms).
- Error rate returned to normal.
- Users can successfully access the API.

## 8. Summary Flow



## 9. Additional Checks (Real World)

- ✓ Check if the issue is only in one region or global.
- ✓ Verify if the problem started after a deployment.
- ✓ Check for unusual traffic patterns.
- ✓ Look for dependency failures (database, cache, external APIs).
- ✓ Review recent configuration changes.
- ✓ Use traces to find the exact slow part of the request.
- ✓ Check if there are resource limits (CPU, memory, connections).

## 10. Key Takeaways

- ✓ Start with the symptom and gather evidence.
- ✓ Use metrics, logs and traces together. Test and eliminate possible causes. Restore service first, then do a deeper investigation.
- ✓ Always verify the fix with real data. Follow a clear and structured troubleshooting approach.
- ✓ Document what happened and prevent it from recurring.

# Root Cause Analysis & Postmortems

A postmortem is a blameless analysis of an incident. It helps you understand what happened, find the **real root cause**, and implement improvements so the same issue does not happen again.

## 1. What is Root Cause Analysis?



- Finds the underlying cause of a problem.
- Looks beyond the symptoms and immediate failures.
- Considers technical, process and human factors.
- Helps prevent the same issue from happening again.

## 2. What is a Postmortem?



- A structured write up after an incident.
- Documents what happened, why it happened, and what will be done to prevent it.
- Focuses on learning and improvement, not blame.
- Shared with the team and stakeholders.

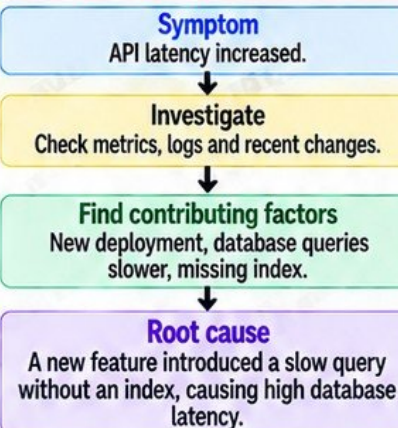
## 3. Key Sections of a Postmortem

- 1 Timeline**  
When things happened and in what order.
- 2 Impact**  
What was affected (users, services, revenue, etc).
- 3 Contributing Factors**  
What conditions made the incident possible.
- 4 Root Cause**  
The underlying cause, not just the symptom.
- 5 Corrective Actions**  
What will be done to prevent recurrence.

## 4. Example Incident Timeline

- 10:00** Deployment started (new version).
- 10:05** Error rate increased.
- 10:10** API latency started to rise.
- 10:15** Users reported issues.
- 10:20** Rollback initiated.
- 10:30** Service fully recovered.

## 5. Finding the Root Cause (Example)



## 6. Corrective Actions (Example)

- ✓ Add missing database index.
- ✓ Add automated performance tests before deployment.
- ✓ Improve monitoring and alerting for query latency.
- ✓ Implement a staged rollout.
- ✓ Update deployment checklist.
- ✓ Add runbook for quick rollback.
- ✓ Review database query performance regularly.

## 7. Why Postmortems Matter



- ✓ Turn incidents into learning opportunities.
- ✓ Help improve systems, processes and tools.
- ✓ Identify gaps in monitoring, testing or design.
- ✓ Reduce the chance of repeat incidents.
- ✓ Build a stronger and more reliable system over time.
- ✓ Encourage a culture of transparency and continuous improvement.

## 8. Blameless Culture



- ✓ Focus on what happened, not who is at fault.
- ✓ Humans make mistakes.
- ✓ Look at system and process improvements.
- ✓ Creates a safe environment for honest discussions.
- ✓ Leads to better solutions and stronger teams.

## 9. Key Takeaways



- ✓ Understand the full timeline of the incident.
- ✓ Assess the real impact.
- ✓ Find the actual root cause.
- ✓ Implement actionable corrective actions.
- ✓ Share and discuss learnings with the team.
- ✓ Postmortems make systems more reliable and prevent repeat incidents.



@veriqta



veriqta



veriqta



veriqta



veriqta



@veriqta\_academy

# Capacity, Saturation & Scaling

Understanding capacity and saturation helps you **prevent outages** before they happen. SREs monitor key resources, plan for traffic growth, and use the right scaling strategy to keep systems reliable and performant.

## 1. Key Resources to Monitor



### CPU

High CPU means the system is working hard.



### Memory

High memory usage can cause slow performance or crashes.



### Disk

Low disk space can stop services from writing logs or data.



### Connections

Too many connections can exhaust limits.



### Queues

Growing queues indicate the system cannot keep up with demand.



### Traffic

Increasing traffic may require scaling before saturation.

## 2. What is Saturation?

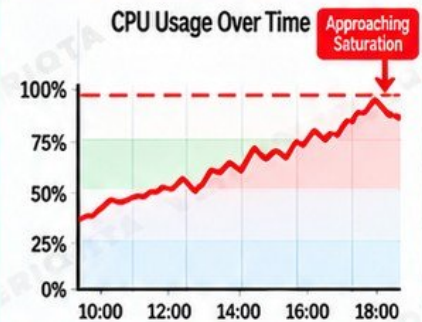
Saturation happens when a resource reaches its maximum capacity, causing slow responses, errors or total failure.



### Signs of saturation:

- High CPU or memory usage
- High disk I/O or disk full
- Longer response times
- Increasing error rates
- Growing queue length
- Dropped connections
- Services becoming unstable

## 3. Example: Approaching Capacity



### What SREs look for:

- Consistent upward trend
- Spikes during peak hours
- Sustained high usage (not just a short spike)
- Reduced headroom (less than 20 to 30%)
- Related increase in errors or latency

## 4. Horizontal vs Vertical Scaling

### Vertical Scaling (Scale Up)



- Increase the resources of a single server (more CPU, memory, disk).
- Simple to implement.
- Has hardware limits.
- Good for moderate growth.

### Horizontal Scaling (Scale Out)



- Add more servers or instances.
- Distributes traffic across multiple instances.
- Handles large traffic growth.
- Improves availability and fault tolerance.

## 5. When to Scale?

- ✓ CPU or memory consistently above 70 to 80%.
- ✓ Increasing response times and latency.
- ✓ Growing queue length.
- ✓ Rising error rates (5xx, timeouts).
- ✓ Traffic growth (more users, new features, seasonal spikes).
- ✓ Reduced headroom (less than 20 to 30% available).
- ✓ Planned events (product launch, marketing campaign).

## 6. Real World Example (E Commerce)



More users during a flash sale



Traffic increases 3x



CPU and database connections increase



Response time increases and errors start



Auto scaling adds more instances and load balancer distributes traffic

## 7. Capacity Planning Tips



- ✓ Understand current usage and trends.
- ✓ Plan for future traffic growth.
- ✓ Set alerts before reaching critical levels.
- ✓ Test scaling during peak load.
- ✓ Consider both application and database capacity.
- ✓ Include external dependencies (APIs, networks).

## 8. Key Takeaways



- ✓ Monitor key resources (CPU, memory, disk, connections, queues).
- ✓ Saturation leads to slow performance and failures.
- ✓ Use the right scaling strategy (vertical or horizontal).
- ✓ Plan for growth and set alerts early.
- ✓ SREs act before users are impacted.
- ✓ Capacity planning is essential for long term reliability.



@veriqta



veriqta



veriqta



veriqta



veriqta



@veriqta\_academy

# Think Like an SRE

An SRE thinks in systems, data and outcomes. They use a structured approach to keep services **reliable**, handle incidents effectively and **continuously improve** the system.

## 1. Observe



- Understand the system.
- Know what good looks like.
- Be aware of dependencies.
- Look for early signs of problems.

Visibility helps you see issues before users are impacted.

## 2. Measure



- Use metrics, logs, traces and alerts.
- Track key indicators such as latency, errors, traffic and resource usage.
- Measure against SLOs and baselines.

You cannot improve what you do not measure.

## 3. Define Reliability



- Set clear SLOs and SLIs.
- Define what is acceptable for your service.
- Understand user impact and business requirements.
- Balance reliability, performance and cost.

Reliability is a feature measured by real user experience.

## 4. Detect



- Use alerts, dashboards and automated checks.
- Detect anomalies early.
- Identify the scope and potential impact.
- Confirm it is a real issue.

Fast and accurate detection reduces the impact on users.

## 5. Investigate



- Gather evidence from metrics, logs and traces.
- Form hypotheses.
- Check the most likely causes first.
- Isolate the root cause.

Follow a logical and methodical approach.

## 6. Mitigate



- Take immediate action to reduce impact.
- Use workarounds or failover if needed.
- Communicate with stakeholders.
- Keep the focus on restoring service.

Stop the bleeding first.

## 7. Recover



- Restore full service.
- Verify the system is stable.
- Monitor closely for recurrence.
- Ensure no hidden issues remain.

Recovery is not just bringing the service back, but making sure it stays healthy.

## 8. Learn & Improve



- Conduct a blameless postmortem.
- Identify the root cause and contributing factors.
- Implement corrective actions.
- Improve monitoring, alerting, runbooks and processes.

Every incident should make the system stronger.

## 9. End to End SRE Mindset



## 10. Real World Scenario (Junior Engineer)



### Scenario:

The API for a web application is suddenly slower. Users are reporting that pages take much longer to load. Error rate is still low, but latency has increased significantly in the last 10 minutes.

### Question:

As a junior engineer, what should you inspect first?

- ☐ A Application logs for errors
- ☐ B Database CPU and query latency
- ☐ C Load balancer configuration
- ☐ D Server disk space



### Explanation:

Start with the service metrics to see where the latency is coming from. Check application metrics to identify which component is slower (for example database, external service or the application itself). Then dive into that component's metrics, logs and traces.

Always follow the evidence. Start with data, not assumptions.



@veriqta



veriqta



veriqta



veriqta



veriqta



@veriqta\_academy